Software Quality Assurance

part one
introduction

chapter one
The software quality challenge

1•1 The uniqueness of software quality assurance

1•2 The environments for which SQA methods are developed

Differences between software and other industrial product (automobiles,radios,washing machines)

- **product complexity**

  Measured by number of operational modes the product permits, industrial product does not allowed more than a few thousand modes of operation, software can find millions of software operation .

- **product visibility**

  Software are invisible but industrial product are visible. Most of defect in an industrial can detected during the manufacturing process, but defect in software are invisible.

- **product development and production process**

  phases at which detecting defect **in industrial product** :

  1- Product development

  designers and SQA staff check and test the product prototype in order to detect defect

Differences between software and other industrial product (automobiles,radios,washing machines) (cont.)

2- product production planning

Production process and tools are designed and prepared, production line may be need to be designed and built.

3- manufacturing

QA procedures are applied to detect defect of products themselves, this can be corrected by change in design or materials or production tools to eliminate defect.

phases at which detecting defect **in software** product :

1- Product development

detect inherent product defects, therefore prototype may be reproduction

2- product production planning

This is not required for software product

3- manufacturing

Manufacturing software is limited to copying the product and printing copies of manuals .detect defect are limited at this phase

Factors affecting defect detection in software products vs. other industrial products

characteristic	Software product	Other Industrial product
Complexity	Usually, very complex product allowing for very large number of operational options.	Degree of complexity much lower, allowing at most a few thousand operational options.
Visibility of product	Invisible product, impossible to detect defects or omissions by sight (e.g. of a diskette or CD storing the software)	Visible product, allowing effective detection of defects by sight.
Nature of development and production process	Opportunities to detect defects arise in only one phase, namely product development.	Opportunities to detect defects arise in all phases of development and production: □ product development □ product production planning □ manufacturing

The uniqueness of the software development process

- High complexity
- Invisibility of the product
- Limited opportunities to detect defects ("bugs")

Environments for which SQA methods are develop:

- pupils and student as part of their education.
- software amateurs develop software as a hobby.
- professional in engineering, economic, management and develop software to assist them in their work in calculation,research,survey.
- software development professionals (system analyst, programmers).

Characteristic of SQA environment process

1• Contractual condition

Contract between software developer and the customer:

- define a list of functional requirement need
- product budget
- project timetable

2• Subjection to customer-supplier relationship

Project team cooperate continuously with the customer, to change any request need

3• Requirement for teamwork

- timetable requirement
- variety of specialization
- benefit from professional

Characteristic of SQA environment process (cont .)

4• cooperation and coordination with other software teams
cooperation may be required with:

- other software development teams in the same organization
- Hardware development teams in the same organization
- software and hardware development teams of other suppliers
- customer software and hardware development teams that take part in the project's development (see figure 1)

5• Need for interfaces with other software systems

types of interfaces:-

- input interface
other software transmit data to our software

- output interface
our system transmit data to other.

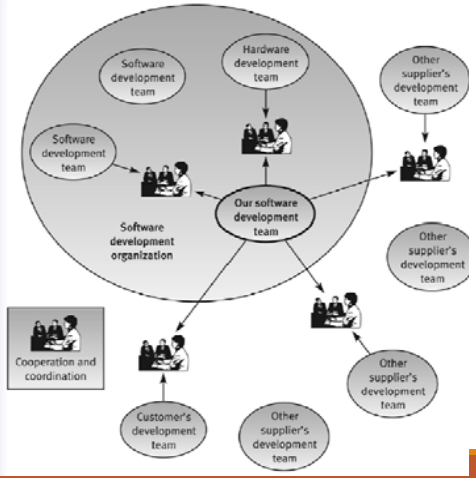
- input-output interface
(EX. salary processing software package -see figure 2 -)

6• Need to continue carrying out a project when team members change

7• Need to conduct maintenance of the software system for several years

Environment Characteristics

Cooperation &coordination scheme for a software development team



- Being contracted
- Subjection to customer supplier relationship
- Requirement for teamwork
- Need for cooperation and coordination with other development teams
- Neioned for interfaces with other softwardinate systems
- Need to continue carrying out a project while the team changes
- Need to continue maintaining the software system for years

Salary software system - an example of software interfaces

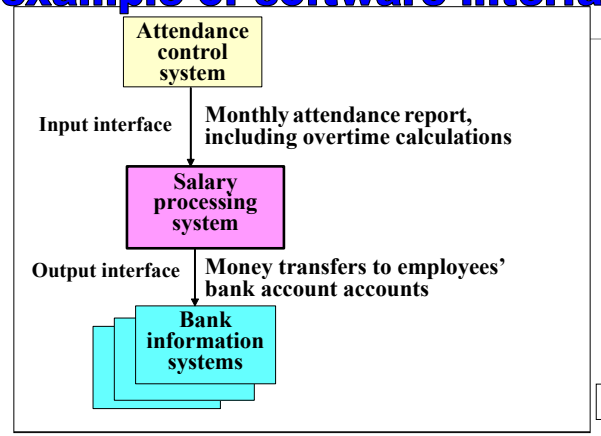


Figure 2

Chapter Two

What is software quality?

Definitions & Terminology

- Software
  - Definition & characterization
  - Why do the classical approach to QA not apply?
- Software quality
  - Definition
  - Quality models
  - Quality attributes
- Measure quality
  - Reliability, Availability, safety, maintainability,
  - Defect counts
  - Software errors, faults and failures
  - Classification of the causes of software errors

Basic Terminology

Software is:

Computer programs, procedures, and possibly associated documentation and data pertaining to the operation of a computer system.

Software - IEEE definition

Software is:

- **Computer programs**
  to activate computer to perform the required application
- **Procedures**
  define the order and schedule that program are performed
- **Documentation**
  for development, users (the requirements report, design reports, program descriptions, etc.)
- **Data** pertaining to the operation of a computer system.

What is special about Software?

- Invisibility of the product
- Limited opportunities to detect defects ("bugs")
- Often new demanding functionality has to be realized
- Often software has to realize extraordinary high complexity

Classical Quality Assurance (Hardware)

- **Requirements:** complete set of external quality Characteristics → complete set of internal quality characteristics and a detailed and complete set of requirements and specifications
- **Design:**
  - Design that satisfies the requirements and specifications
  - design for reliability (wear out)
  - design for manufacturability
  - design for maintainability (e.g., self-diagnosis)
- **Manufacturing:** statistical production process control with acceptance sampling
- **Operation:** collect failure data for continuous improvement and predictions (intelligent maintenance)

Software Quality Assurance?

- **Requirements:**
  - Completeness is hard to achieve (complexity)
- **Design:**
  - design for reliability only in special cases
  - design for manufacturability is not required
  - design for maintainability in special cases
  - Focal area of software quality assurance!
- **Manufacturing → Implementation:**
  - limited success with statistical process control (metrics)
- **Operation:**
  - Fixing found bugs

What is quality?

- Quality, simplistically, means that a product should meet its specification.
- This is problematical for software systems
  - There is a tension between customer quality requirements (efficiency, reliability, etc.) and developer quality requirements (maintainability, reusability, etc.);
  - Some quality requirements are difficult to specify in an unambiguous way;
  - Software specifications are usually incomplete and often inconsistent.

Approaches to Tackle Quality

- **Transcendental view**: quality is universally identifiable, absolute, unique and perfect
- **Product view**: the quality of a product is measurable in an objective manner
- **User view**: quality is fitness for use
- **Manufacturing view**: quality is the result of the right development of the product
- **Value-based view (Economic)**: quality is a function of costs and benefits

Software Quality – IEEE View

Software quality is:
(1) The degree to which a system, component, or process meets specified requirements.
(2) The degree to which a system, component, or process meets customer or user needs or expectations.

Software Quality

Software quality **is** Conformance to explicitly stated functional and performance requirements, explicitly documented development standards, and implicit characteristics that are expected of all professionally developed software.

[Pressman2004]

Software Quality

Quality – the degree of excellence of something.
We measure the excellence of software via a set of attributes.

(≠ "satisfying requirements")!

- Quality is absolute or at least independent of the requirements underlying the product.
- It is part of the software development to "get the right requirements".

(≠ "user satisfaction")!

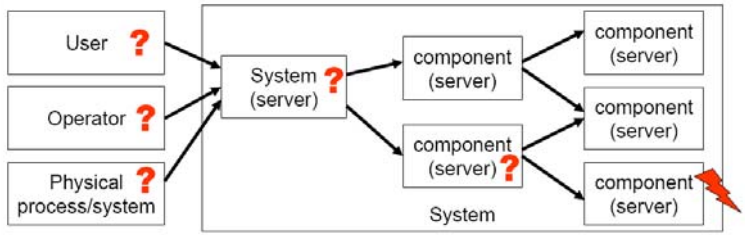
- McDonald's restaurants are popular, but …

Faults, Errors and Failures

- Error is a defect within the system. (fault cause)
- An **error** is a derivation of the required operation of the system or subsystem. (Manifestation of a fault in a system)
- **software faults** are a human action which results in software containing a error
- A **system failure** occurs when the system fails to perform its required function. (Transition to incorrect service delivery)
- If the distinction between fault and failure is not critical, **defect** can be used as a generic term.

Recursive Nature of Faults

- A fault in a component can "infect" all other components which depend on it.
- Chain of Faults/Failures



Fault/Failure Chain

Error → Fault → Failure

error→ fault

- an error is *latent* when it has not been recognized
- an error is *detected* by a detection algorithm/mechanism

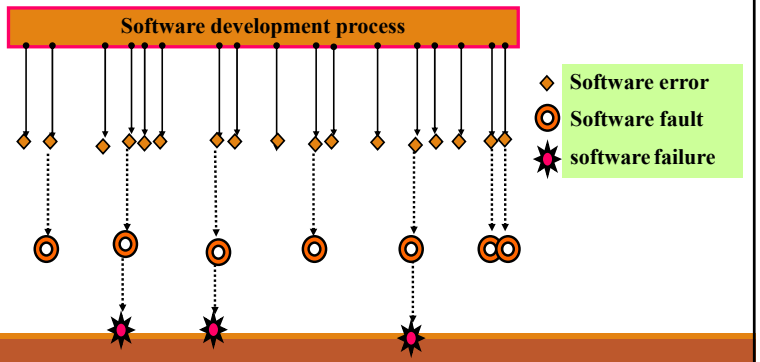
fault → failure

- a fault which has not been activated by the computation process is *dormant*
- a fault is *active* when it produces an failure

Failure → fault

- a failure occurs when an error "passes through" and affects the service delivered
- a failure results in a fault for the system which contains or interacts with the component

- The root of any software failure is a software error
- Not all software errors become software faults.
- The software error can cause improper functioning.
- Not all software faults end with software failure.



Examples for Fault/Failure Chain

Program error (software):

- a dormant *fault* in the written software (instruction or data)
- upon activation the fault becomes active and produces an *error* (system state)
- if the erroneous data affects the delivered service, a *failure* occurs

Electromagnetic interference (hardware):

- leads to *faulty* input value (either digital or analog)
- by reading the input the fault becomes active and produces an *error*
- if the erroneous input value is processed and becomes visible at the interface

  a *failure* occurs

Examples for Fault/Failure Chain

Program error (software):

- a dormant *fault* in the written software (instruction or data)
- upon activation the fault becomes active and produces an *error* (system state)
- if the erroneous data affects the delivered service, a *failure* occurs

Electromagnetic interference (hardware):

- leads to *faulty* input value (either digital or analog)
- by reading the input the fault becomes active and produces an *error*
- if the erroneous input value is processed and becomesvisible at the interface

  a *failure* occurs

Quality Models

- Such general definitions of software quality are not sufficient in practice
- Thus, software quality is described by specific quality models
- causal relationship from intangible quality views to tangible software measures“

Two main approaches:

- Standard Models:
  - McCall
  - ISO/IEC 9126
- Application or company specific quality models
  - FURPS
  - GQM Approach

Chapter Three

Software quality factors

- **The need for comprehensive software quality requirements**
- **Classification of requirements into software quality factors**
- **Product operation factors**
- **Product revision factors**
- **Product transition factors**
- **Alternative models of software quality factors**
- **Who is interested in defining quality requirements?**
- **Software compliance with quality factors**

The need for comprehensive software quality requirements

There are some characteristic common to all requirement fail:

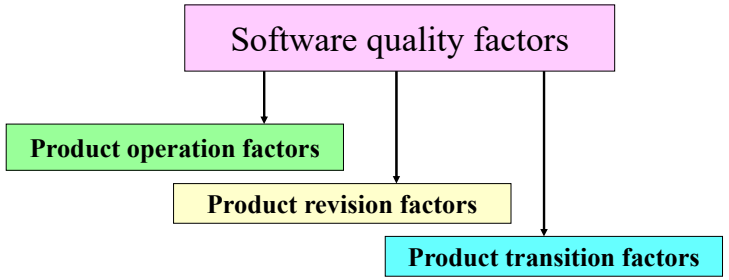
1- All the software suffered from poor performance in important area such maintenance reliability ,software reuse ,training.

2- The cause for the poor performance of the developed software projects in these areas was the lack of predefined requirement.

3- All the software projects satisfactorily fulfilled the basic requirement for correct calculations

Classification of software requirement into software quality factors

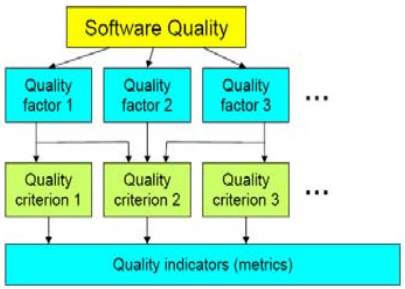
McCall's software quality factors model



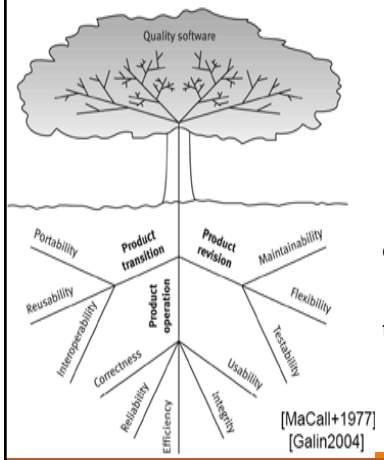
Factor-Criteria-Metrics-Model

Classification into :

- **Factors** (to specify): They describe the external view of the software, as viewed by the users.
- **Criteria** (to build): They describe the internal view of the software, as seen by the developer.
- **Metrics** (to control): They are defined and used to provide a scale and method for measurement.



McCall's Factor Model Tree



[MaCall+1977]
[Galin2004]

- A **quality factor** represents a behavioral characteristic of the system.
  - Operation
  - Revision
  - Transition
- A **quality criterion** is an attribute of a quality factor that is related to software production and design.
- A **quality metric** is a measure that captures some aspect of a quality criterion.

Product operation factors

- **Correctness**
  Correctness requirement are define in a list of the software system"s required outputs.
- **Reliability**
  deal with failures to provide service
- **Efficiency**
  deal with the hardware resources needed to perform all the functions of the software system
- **Integrity**
  Deal with software system security and requirement to prevent access to unauthorized persons
- **Usability**
  Deal with scope of staff resources needed to train and operate the software system

Product revision factors

- **Maintainability**
  Determine the efforts that will be needed by users and maintenance personnel to identify the reasons for software failure
- **Flexibility**
  The capabilities and efforts required to support adaptive maintenance activities
- **Testability**
  Deal with the testing of an information system as well as with its operation

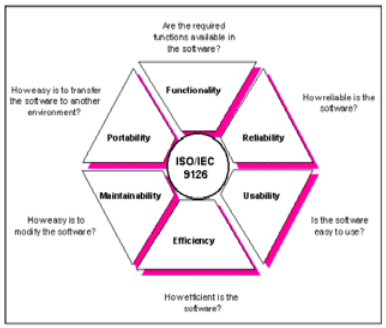
Product transition factors

- **Portability**
  Tend to the adaptation of a software system to other environments consisting of different hardware and operating system
- **Reusability**
  Deal with the use of software modules originally for one project in a new software project currently being developed
- **Interoperability**
  Focus on creating interface with other software or with other equipment

Alternative models of software quality factors

Verifiability:
Define design & programming features that enable efficient verification of the design & programming
Expandability:
Refer to future efforts that will be needed to serve larger populations, improve or add new service
Safety:
Eliminate condition hazardous to operators of equipment as a result of errors in process control software.
Manageability:
Refer to the administrative tools that support software modification during the software development & maintenance periods.
Survivability:
Refer to the continuity of service. these define the minimum time allowed between failures of the system,& the maximum time permitted for recovery of service.

The Six Quality Characteristics of a Software



- **Software quality:** *The totality of features and characteristics of a software product that bear on its ability to satisfy stated or implied needs.*
- **Software quality characteristic**

A set of attributes of a software product by which its quality is described and evaluated. A software quality characteristic may be refined into multiple levels of sub characteristic

- Each characteristic is refined to a set of sub-characteristics
- Each sub-characteristic is evaluated by a set of metrics.
- Some metrics are common to several sub-characteristics.

Characteristics	Subcharacteristics	Definitions
Functionality	Suitability	Attributes of software that bear on the presence and appropriateness of a set of functions for specified tasks.
	Accurateness	Attributes of software that bear on the provision of right or agreed results or effects.
	Interoperability	Attributes of software that bear on its ability to interact with specified systems.
	Compliance	Attributes of software that make the software adhere to application related standards or conventions or regulations in laws and similar prescriptions.
	Security	Attributes of software that bear on its ability to prevent unauthorized access, whether accidental or deliberate, to programs or data.
Reliability	Maturity	Attributes of software that bear on the frequency of failure by faults in the software.
	Fault tolerance	Attributes of software that bear on its ability to maintain a specified level of performance in case of software faults or of infringement of its specified interface.
	Recoverability	Attributes of software that bear on the capability to re-establish its level of performance and recover the data directly affected in case of a failure and on the time and effort needed for it.
Usability	Understandability	Attributes of software that bear on the users' effort for recognizing the logical concept and its applicability.
	Learnability	Attributes of software that bear on the users'effort for learning its application.
	Operability	Attributes of software that bear on the users'effort for operation and operation control.

Characteristics	Subcharacteristics	Definitions
Efficiency	Time behaviour	Attributes of software that bear on response and processing times and on throughput rates in performances its function.
	Resource behavior	Attributes of software that bear on the amount of resource used and the duration of such use in performing its function.
Maintainability	Analyzability	Attributes of software that bear on the effort needed for diagnosis of deficiencies or causes of failures, or for identification of parts to be modified.
	Changeability	Attributes of software that bear on the effort needed for modification, fault removal or for environmental change.
	Stability	Attributes of software that bear on the risk of unexpected effect of modifications.
	Testability	Attributes of software that bear on the effort needed for validating the modified software.
Portability	Adaptability	Attributes of software that bear on the opportunity for its adaptation to different specified environments without applying other actions or means than those provided for this purpose for the software considered.
	Installability	Attributes of software that bear on the effort needed to install the software in a specified environment.
	Conformance	Attributes of software that make the software adhere to standards or conventions relating to portability.
	Replaceability	Attributes of software that bear on opportunity and effort using it in the place of specified other software in the environment of that software.

Hewlett Packard: F.U.R.P.S.

- Result of a statistical project survey at Hewlett Packard 1987

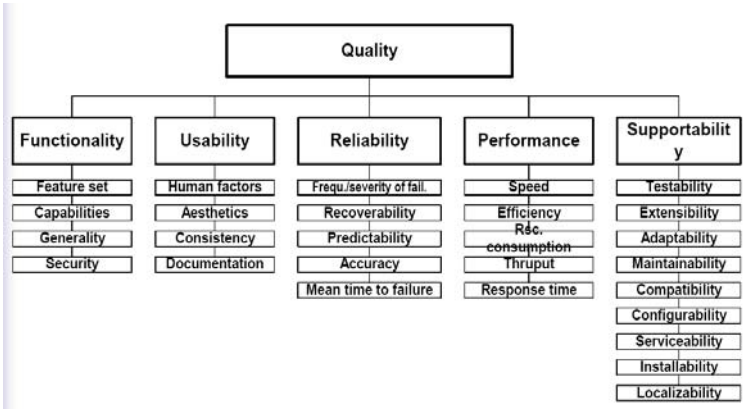
to improve its products:

Factors:

- **F**unctionality: functions it performs, their generality and Security
- **U**sability: aesthetics, consistency, documentation
- **R**eliability: frequency and severity of failure, accuracy of output
- **P**erformance: response time, resource consumption
- **S**upportability: can it be extended, adapted, corrected?

- FURPS is originally a company specific quality model

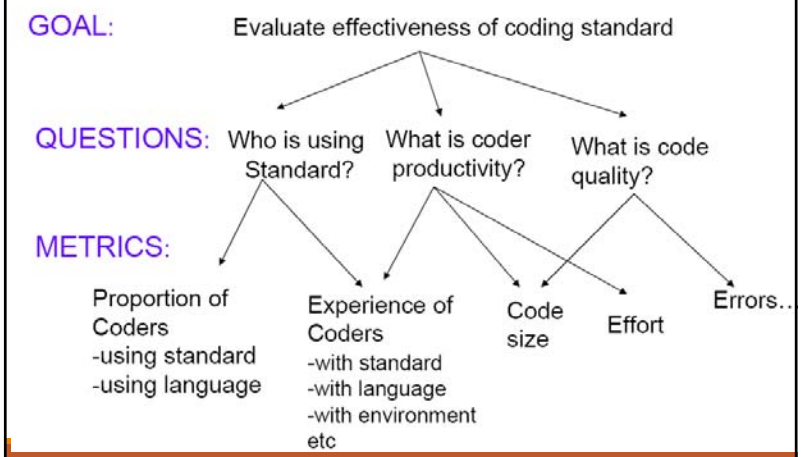
Hewlett Packard: F.U.R.P.S.



GQM: Goal-Question-Metric

- A measurement program can be more successful if designed with the goals in mind.
- GQM approach provides a framework with 3 steps:
  1. List the major goals of the development/maintenance project
  2. Derive from each goal the questions that must be answered to determine if the goals are being met
  3. Decide what must be measured to answer the questions adequately

Example



GQM Discussion

- **Benefits :**
  - generates only those measures relevant to the goal
  - Several measurements may be needed to answer a single question.
  - A single measurement may apply to more than one question.
  - The goal provides the purpose for collecting the data.
- **Not evident from the GQM**
  - The model needed to combine the measurement in a sensible way so that the question can be answered.
  - It must be supplemented by one or more models that express the relationship among the metrics. (equation definition is not clear)
- **Disadvantages:**
  - Additional efforts to derive the goals and metrics
  - Error prone compared to standard models

Measuring Quality?

- User's view:
  - Reliability (number of failures over time)
  - Availability
  - Usability etc.
  - Often directly measurable
- Manufacturer's view:
  - Defect counts
  - Rework costs

Reliability

Reliability is the probability of a component, or system, functioning correctly over a given period time under a given set of operating conditions.

Quantitative:

Reliability *R*(*t*) is the probability that the system will conform to its specification throughout a period of duration *t*.

- Note that t is important
- If a system only needs to operate for ten hours at a time, then that is the reliability target.

Availability

The **availability** of a system is the probability that the system will be functioning correctly at any given time.

Quantitative:

Availability *A* (or *V*) is the percentage of time for which the system will conform to its specification.

- Literally, readiness for service

Safety

- **Safety** is a property of a system that it will not endanger human life or the environment.
- A **safety-related system** (**safety-critical system**) in one by which the safety of equipment or plant is assured.

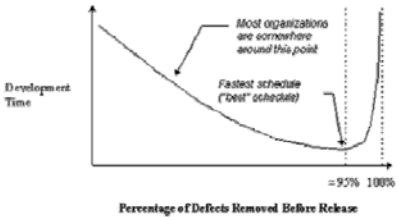
Maintainability

- **Maintenance** is the action taken to retain a system
  in, or return a system to, its designed operating conditions. **Maintainability** is theability of a system to be maintained (ability to undergo repairs and modifications).
- Mean time to repair (MTTR)

Problem: Maintenance induced failures

Defect Counts

- „software errors are the cause of poor software quality“
- In software, higher quality (in the form of lower defect rates) and
  reduced development time go hand in hand.
- Unlike the low-defect kind of quality, attention to this kind of
  quality tends to lengthen the development schedule.



- Intuitively, the occurrence of defects is negatively related to functionality and reliability.
  Defects also interfere, to some degree, with other dimensions of quality.

Relative Cost of Correcting Defects

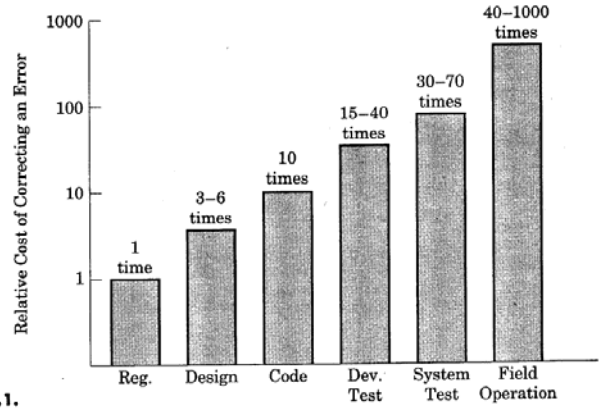
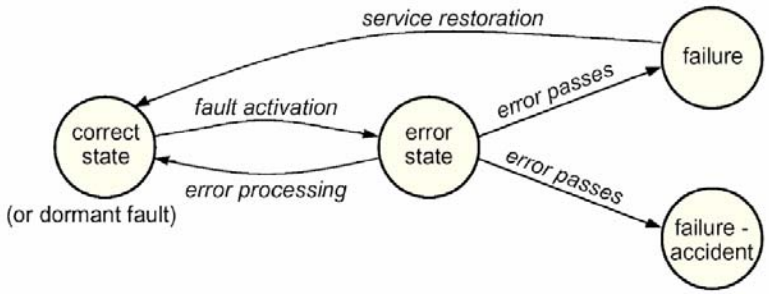


FIGURE 8.1.
Relative cost of correcting an error.

Fault/Failure state transitions



Fault Classification

Nature (Critical distinction):
- random/hardware faults
- logical/systematic/design faults

E.g., Degradation (wear-out) vs. design faults

Duration:
- permanent, transient, intermittent

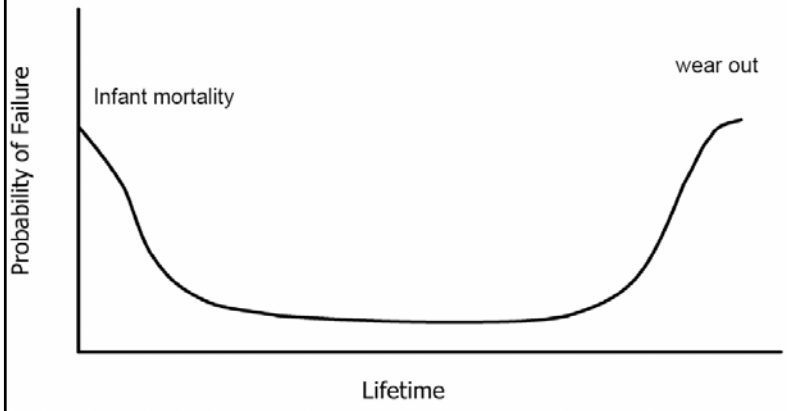
Extent:
- localized, global

Nature: Degradation Faults

- The system used to work but now it does not
- Something broke for some reason:
  - Infant mortality
  - End of useful life—wear out
  - Physical damage—vibration, humidity, temperature

Examples:
- Light bulb burns out
- Disk head crashes
- Power supply fails

Nature: Failure "Bathtub Curve"



Nature: Design Faults

- The system never worked
- Nothing broke
- The design is flawed

Examples:

- Incorrect electrical wiring
- **Software defect!**

Dealing With Faults

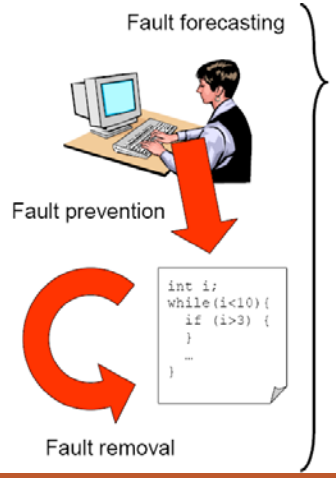
Fault Prevention:
- Development techniques that prevent the introduction of faults

Fault Elimination:
- Development techniques that find and remove faults already introduce

Fault Forecasting:
- Estimate current number, future incidence and likely consequences

Fault Tolerance (not considered here):
- Execution-time techniques that cope with the effects of faults

Fault Avoidance



Activities:
- Preventing the introduction or occurrence of faults **(fault prevention):**
  - SWT, formal methods, high leve languages, CASE tools, …
- Reducing the number and seriousness of faults **(fault removal):**
  - Analysis ⇨ diagnosis ⇨ correcti
- Evaluating the present numbe future incidents, and severity faults (**fault forecasting):**
  - Statistics & management

Main causes for software faults:

1. Faulty requirements definition
2. Client-developer communication failures
3. Deliberate deviations from software requirements
4. Logical design errors
5. Coding errors
6. Non-compliance with documentation and coding instructions
7. Shortcomings of the testing process
8. User interface and procedure errors
9. Documentation errors

The Cost of Quality

- *Cost of Quality* includes all costs incurred in the pursuit of quality or in performing quality related activities such as appraisal costs, failure costs and external failure costs.

[Pressman2004]

The Quality Compromise:
- We cannot wait for specifications to improve before paying attention to quality management.
- We must put quality management procedures into place to improve quality in spite of imperfect specification

(1) Quality Assurance

Quality Assurance [ISO 9000]:
All those planned and systematic actions necessary to provide adequate confidence that a product or service will satisfy requirements for quality

Software quality assurance [IEEE]:
1. A planned and systematic pattern of all actions necessary to provide adequate confidence that an item or product conforms to established technical requirements.
2. A set of activities designed to evaluate the process by which the products are developed or manufactured.
   Contrast with: quality control.

Quality Assurance

Software quality assurance is :
A systematic, planned set of actions necessary to provide adequate confidence that the software development process or the maintenance process of a software system product conforms to established functional technical requirements as well as with the managerial requirements of keeping the schedule and operating within the budgetary confines.

- *Quality assurance* consists of the auditing and reporting functions of management

(2) Quality Planning

- ***Quality planning*** is the process of assessing the requirements of the procedure and of the product and the context in which these must be observed.

- ***Quality assurance plan*** is the central aid for planning and checking the quality assurance.

Quality Assurance Plan

- A quality assurance plan sets out the desired product qualities and how these are assessed and defines the most significant quality attributes.

- The quality assurance plan should define the quality assessment process.

- It should set out which organizational standards should be applied and, where necessary, define new standards to be used.

Quality Assurance Plans

- Quality assurance plan structure:
- Product introduction
- Product plans
- Process descriptions
- Quality goals
- Risks and risk management
- Quality assurance plans should be
- short,
- succinct

(If they are too long, no-one will read them)

Example: SQA Plan

Purpose of Plan

- References
- Management
- organization structure, SQA tasks, their placement in the process
- roles and responsibilities related to product quality
- Documentation
- project documents, models, technical documents, user documents.
- Standards, Practices and Conventions
- Reviews and Audits
- Test
- test plan and procedure
- Problem Reporting and Corrective action
- Tools, Techniques and Methodologies
- Code Control
- Media Control
- Supplier control
- Records Collection, Maintenance and Retention
- Training
- Risk Management

(3) Quality Control

Quality Control [ISO 9000]:

The operational techniques and activities that are used to fulfill requirements for quality

- *Quality Control* is the series of inspections, reviews and tests used throughout the development cycle to ensure that each work product meets the requirements placed upon it.

Quality Control

- This involves checking the software development process to ensure that procedures and standards are being followed.
- There are two approaches to quality control
  - Quality reviews;
  - Automated software assessment and software measurement.

Quality Control

Objective:
- minimize the produced defects
- increase the product quality

Implementation approaches:
- Fully automated
- Entirely manual
- Combination of automated tools and human interactions

Quality Control

Quality control includes a ***feedback loop*** to the process:
- provide management with the necessary data about product quality.
- gain the insight and confidence of product quality

Two types of quality control:
- **Quality design**: the characteristics that designers specify for an item (includes: requirements, specifications, and the design of the system).
- **Quality of conformance**: the degree to which the design specification are followed. It focuses on implementation based on the design.

Quality Assurance System

- *Quality assurance system* is the organizational structure, responsibilities, procedures, processes and resources for implementing quality management.

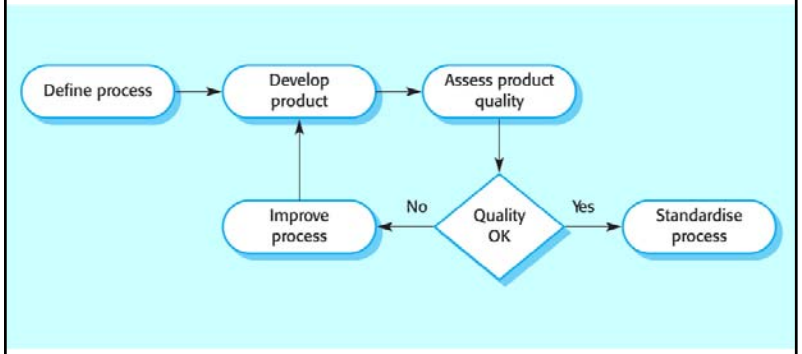
Process and Product Quality

- The quality of a developed product is influenced by the quality of the production process.
- This is important in software development as some product quality attributes are hard to assess.
- However, there is a very complex and poorly understood relationship between software processes and product quality.

Process-based Quality

- There is a straightforward link between process and product in manufactured goods.
- More complex for software because:
  - The application of individual skills and experience is particularly important in software development;
  - External factors such as the novelty of an application or the need for an accelerated development schedule may impair product quality.
- Care must be taken not to impose inappropriate process standards these could reduce rather than improve the product quality.

Process-based quality



Practical Process Quality

- Define process standards such as how reviews should be conducted, configuration management, etc.
- Monitor the development process to ensure that standards are being followed.
- Report on the process to project management and software procurer.
- Don't use inappropriate practices simply because standards have been established.